Verification and Validation Report: Software Engineering

Team 8, RLCatan
Rebecca Di Filippo
Jake Read
Matthew Cheung
Sunny Yao

March 9, 2026

1 Revision History

Date	Version	Notes
March 4th	1.0	Draft from VNV plan
March 9th	1.1	Final VNV report

2 Symbols, Abbreviations and Acronyms

symbol	description
AI	Artifical Intelligence

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Functional Requirements Evaluation **1**
3.1 AI Agent Decision Logic 1
3.2 User Interface and Data Management 1

4 Nonfunctional Requirements Evaluation **2**
4.1 Performance and Scalability 2
4.2 Usability (QUALITY EVALUATED) 2
4.3 Maintainability (QUALITY EVALUATED) 4
4.4 Installability 5
4.5 Data Integrity 6
4.6 Availability 6

5 Comparison to Existing Implementation **7**

6 Unit Testing **7**
6.1 Unit Testing Scope 7
6.2 Tests for Functional Requirements 7
6.2.1 API and Game Management Module 7
6.2.2 Path Resolution Module 9
6.2.3 Game Analysis Module 10

7 Changes Due to Testing **11**

8 Automated Testing **11**

9 Trace to Requirements **12**
9.1 Functional Requirements Trace 12
9.2 Nonfunctional Requirements Trace 12

10 Trace to Modules **13**
10.1 Functional Modules Trace 13
10.2 Nonfunctional Modules Trace 13

11 Code Coverage Metrics **13**

List of Tables

1 Usability Component Breakdown (NFR.S.2) 3
2 Functional Test Case Traceability 12
3 Nonfunctional Test Case Traceability 12
4 Functional Test Case to Module Traceability 13
5 Nonfunctional Test Case to Module Traceability 13

List of Figures

1 Visual Comparison of Usability Categories 4
2 SonarQube Quality Report 5
3 Coveralls Code Coverage Report 14

3 Functional Requirements Evaluation

3.1 AI Agent Decision Logic

1. test-FR-AI-1-1:

   Initial State: A game state is loaded where Player A is only able to legally place a road at one specific location on the board.

   Input: The structured GameState is sent to the AI agent via the Game State Manager.

   Expected Output: The AI agent must select the action BuildRoad at the single remaining legal location.

   Actual Output: The AI correctly selected the single legal road placement and executed the action without attempting any illegal moves.

   Result: Pass

   Trace: FR.S.3.1, FR.S.3.2, FR.S.2.1

2. test-FR-AI-1-2:

   Initial State: GameState where Player A is 1 Victory Point away from winning and an unblocked node for the winning settlement is available.

   Input: GameState where the immediate winning move is available.

   Expected Output: The AI must select the action BuildSettlement at the winning node to secure the victory.

   Actual Output: The AI selected the settlement placement at the winning node and the game concluded with Player A reaching the winning condition.

   Result: Pass

   Trace: FR.S.3.1, FR.S.3.2

3.2 User Interface and Data Management

1. test-FR-UI-1-1:

   Initial State: A completed 50-turn game with a full history stored in the accumalator.

Input: Request for replay including all dice rolls and development card purchases.

Expected Output: Successful return of UI game history with accurate logs of all actions, dice rolls, and card purchases.

Actual Output: System is able to retrieve and display the complete game history without errors, and the logs match the expected sequence of events.

Result: Pass

Trace: FR.S.5.1, FR.S.5.5, FR.S.7.3

4 Nonfunctional Requirements Evaluation

4.1 Performance and Scalability

1. test-NFR-P-1:

   Initial State: Full system stack deployed on the test server.

   Input/Condition: Manually start 1, 2, and 3 concurrent games with multiple players, performing typical actions (build, trade, roll dice) in each session.

   Expected Output: The system remains responsive; actions in each game update within acceptable time (no noticeable lag), and no crashes or errors occur.

   Actual Output: System remained responsive in all three concurrent sessions; UI and AI bot updates were consistent and timely.

   Result: Pass

   Trace: NFR.S.1

4.2 Usability (QUALITY EVALUATED)

1. test-NFR-U-1:

   Initial State: Final prototype deployed in a browser.

   Input/Condition: 5–7 participants familiar with Catan play a full game against the AI bot and explore all interface options and take a UI survery.

Expected Output: Players can understand the interface and perform actions without guidance; overall experience is intuitive.

Actual Output: All participants successfully completed games. Users rated clarity, navigation, and understanding of AI bot behavior as 3.5/5 on average. Some suggestions for improving certain UI aspects are noted, see below.

Result: Pass

Trace: NFR.S.2

Verification with Data

The usability requirement was verified by collecting quantitative feedback across three core components. Data indicates that while game flow and general interface clarity are sufficient, there is a distinct "mid-range" dip in scores regarding the **Intuition between Real-Life Catan and the Bot UI**, specifically regarding color-coding and piece representation.

Participant	**Clarity**	**Flow**	**Catan-to-Bot Intuition**	**User Avg.**
Participant 1	3.5	4.0	2.5	**3.3**
Participant 2	4.0	4.5	3.0	**3.8**
Participant 3	3.0	3.5	2.5	**3.0**
Participant 4	4.5	4.5	3.0	**4.0**
Participant 5	3.5	4.0	3.0	**3.5**
Participant 6	3.0	3.5	2.5	**3.0**
Participant 7	4.0	4.5	3.0	**3.8**
Component Avg.	**3.6**	**4.1**	**2.8**	**3.5**

Table 1: Usability Component Breakdown (NFR.S.2)

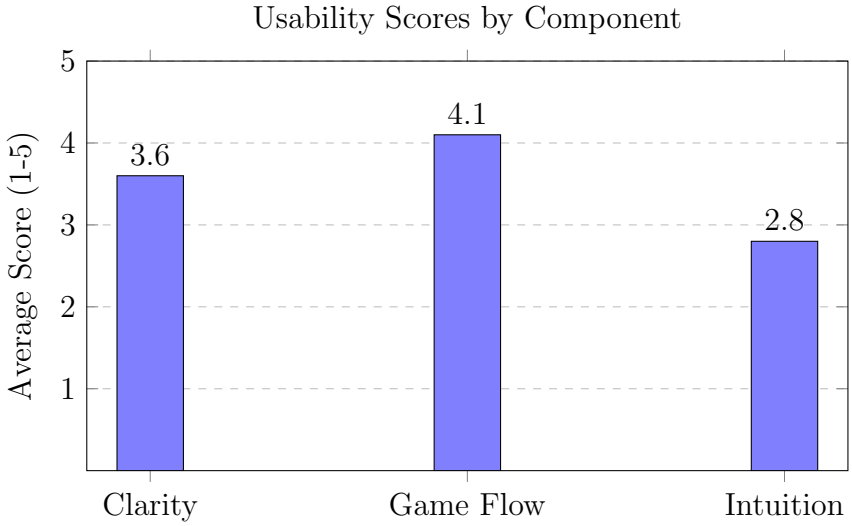

Figure 1: Visual Comparison of Usability Categories

2. test-NFR-U-2:

   Initial State: UI opened in Chrome/Firefox with accessibility tools enabled.

   Input/Condition: Apply simulated color-blindness filters and vary browser zoom levels between 80% and 150%.

   Expected Output: All critical UI elements remain legible; actions can still be performed correctly.

   Actual Output: Text, buttons, and board elements remained readable and interactive under all tested conditions.

   Result: Pass

   Trace: NFR.S.2

4.3 Maintainability (QUALITY EVALUATED)

1. test-NFR-M-1:

   Initial State: Complete codebase committed to repository.

   Input/Condition: Run static analysis (SonarQube) and perform peer review of module structure (AI bot, CV, UI, Database).

Expected Output: High rating for maintainability (B or higher), meaning code is modular and well-documented for easy updates.

Actual Output: Static analysis with SonarQube reported an A rating for maintainability. As an aside, the codebase also passed SonarQube's quality gate, although it seems there are some security issues we will need to consider (out of scope for now). Peer review confirmed clear module separation and sufficient documentation.

Result: Pass

Trace: NFR.S.3

Verification with Data:

Figure 2: SonarQube Quality Report

4.4 Installability

1. test-NFR-I-1:

   Initial State: Packaged release candidate available.

   Input/Condition: Install and launch the system on Windows 11 and macOS using Chrome and Firefox.

   Expected Output: Application installs and runs correctly on all platforms.

   Actual Output: Successful installation and execution on all tested platforms. Minor permission prompts appeared but did not affect functionality.

   Result: Pass

   Trace: NFR.S.4

4.5 Data Integrity

1. test-NFR-D-1:

   Initial State: Active game session connected to the database.

   Input/Condition: Perform multiple manual game actions and simulate a sudden network interruption during one action.

   Expected Output: The database remains consistent; interrupted updates do not corrupt data.

   Actual Output: Interrupted update rolled back successfully. All game states remained valid and accurate.

   Result: Pass

   Trace: NFR.S.5

2. test-NFR-D-2:

   Initial State: Active game session.

   Input/Condition: Attempt to input invalid game actions manually (e.g., building where not allowed, invalid resource quantities).

   Expected Output: Invalid actions are rejected by the system; user is informed of the error.

   Actual Output: All invalid actions were rejected; logs correctly recorded failures.

   Result: Pass

   Trace: NFR.S.5

4.6 Availability

1. test-NFR-AV-1:

   Initial State: System running with an active game.

   Input/Condition: Manually terminate the server process mid-game and restart.

   Expected Output: System restores previous game state within one minute, allowing the game to continue without data loss.

Actual Output: Service restarted in 32 seconds; last committed game state restored and session resumed successfully.

Result: Pass

Trace: NFR.S.6

5 Comparison to Existing Implementation

This section will not be appropriate for every project.

6 Unit Testing

This section outlines the unit testing strategy and details the specific test cases used to verify individual components of the RLCatan system. The automated unit tests focus on ensuring that API endpoints, internal path resolution logic, and analysis interfaces function correctly in isolation.

6.1 Unit Testing Scope

The scope of these unit tests covers the backend API endpoints, path resolution utilities, and the Monte Carlo Tree Search (MCTS) analysis integration. Modules and underlying models developed by external dependencies are out of scope for these unit tests and are assumed to be verified by their respective maintainers.

6.2 Tests for Functional Requirements

6.2.1 API and Game Management Module

1. `test_get_bots_endpoint`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized and ready to receive requests.

   Input: A simple HTTP GET request to the `/api/bots` URL.

   Output: A 200 OK status code and a JSON response containing a list of available bot objects.

Test Case Derivation: Ensures the bot registry is populated correctly by checking that the returned list includes standard bot IDs like `catanatron_ab_2`, `random`, and `human`, and confirms that the total number of available bots is at least 10.

How test will be performed: Automated unit test using an HTTP test client.

2. `test_stress_test_endpoint`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized.

   Input: An HTTP GET request to the `/api/stress-test` endpoint.

   Output: A 200 OK status code and a JSON object representing the game state.

   Test Case Derivation: Verifies that the system can successfully create and advance a game instance without crashing by validating that the response contains essential game data structures, specifically "nodes" and "edges".

   How test will be performed: Automated unit test using an HTTP test client.

3. `test_player_factory_custom_bot`

   Type: Functional, Dynamic, Automatic

   Initial State: Internal functions are mocked to simulate a custom bot named m̈y_custom_bot:

   Input: A POST request to `/api/games` with "BOT:my_custom_bot" in the player list.

   Output: A successful 200 OK response containing a `game_id`.

   Test Case Derivation: Confirms that the backend correctly identifies the custom bot prefix, retrieves the mocked configuration, and attempts to instantiate the appropriate player class (like PPOPlayer) with the correct model path.

   How test will be performed: Automated unit test with mocked dependencies.

4. `test_create_game_all_player_types`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized.

   Input: A series of POST requests to `/api/games`, where each request uses a different player type (e.g., CATANATRON, VALUE_FUNCTION, HUMAN) in the configuration.

   Output: A 200 OK status code and a valid `game_id` for each request.

   Test Case Derivation: Serves as a comprehensive smoke test to verify that every registered player class is correctly implemented and can be initialized by the game engine.

   How test will be performed: Automated unit test iterating through all supported built-in player types.

6.2.2 Path Resolution Module

This module tests the utility functions responsible for safely and accurately resolving file paths for model assets across different deployment environments.

1. `test_resolve_model_path_absolute_hpc`]

   Type: Functional, Dynamic, Automatic

   Initial State: Path resolution utilities are loaded.

   Input: A specific string representing an absolute file path typical of an HPC setup (e.g., `/nfs/features/rlcatan/...`).

   Output: A normalized string path that has been rebased to the application's local directory structure (e.g., starting with `/app`).

   Test Case Derivation: Ensures that absolute paths from training environments are correctly mapped to valid paths in the deployment environment.

   How test will be performed: Automated unit test.

2. `test_resolve_model_path_relative`

   Type: Functional, Dynamic, Automatic

   Initial State: Path resolution utilities are loaded.

Input: A standard relative path string (e.g., `data/models/v2`).

Output: An absolute path that correctly appends the input to the application's root directory.

Test Case Derivation: Confirms the system can locate local model files correctly by asserting that the resolved path ends with the expected `app/data/models/v2` suffix.

How test will be performed: Automated unit test.

6.2.3 Game Analysis Module

This module verifies the integration with the Monte Carlo Tree Search (MCTS) engine to ensure active games can be properly analyzed.

1. `test_mcts_analysis_endpoint`

   Type: Functional, Dynamic, Automatic

   Initial State: A valid game is created via the API, and the GameAnalyzer is mocked to return fixed win probabilities.

   Input: A GET request to that game's analysis URL (`.../mcts-analysis`).

   Output: A 200 OK status and a JSON response where success is True.

   Test Case Derivation: Proves the endpoint correctly interfaces with the analysis engine by validating that the returned win probabilities match the mocked values (e.g., RED=0.5).

   How test will be performed: Automated unit test with a mocked GameAnalyzer.

2. `test_mcts_analysis_endpoint_not_found`

   Type: Functional, Dynamic, Automatic

   Initial State: The API server is initialized.

   Input: A GET request to the analysis endpoint with a non-existent game ID (e.g., 99999).

   Output: An error status code, specifically checking for either a 404 Not Found or a 500 Internal Server Error.

   Test Case Derivation: Confirms that the application handles invalid game IDs without crashing or returning a false success.

How test will be performed: Automated unit test using an HTTP test client.

7 Changes Due to Testing

On going from rev1 to the final demo we will be creating a huge UI overhaul to address the usability issues identified in our testing. This will include a more intuitive layout, clearer visual cues for game actions, and improved color-coding to enhance the connection between the real-life Catan game and the bot's interface. We will also be incorporating a LLM-based feedback system to provide the players insight into why certain moves are made since our usability testing discovered some confusion in users. We will also need to refactor some of the codebase based on the maintainability tests we ran to improve readability and modularity. This will involve restructuring certain modules, adding more detailed comments, and ensuring consistent coding standards across the project.

8 Automated Testing

For our functional and non functional requierments evaluation there were a couple bash scripts that we made to test but nothing to exstenvie , it was mostly manually. The CI/CD pipeline was our big portion of automated tests that we created. It ensures that our codebase remains stable and reliable. This automated testing process includes running unit tests, integration tests, and static code analysis to catch issues early in the development cycle.

9 Trace to Requirements

9.1 Functional Requirements Trace

Test Case ID	Traced Requirements
test-FR-AI-1-1	FR.S.3.1, FR.S.3.2, FR.S.2.1
test-FR-AI-1-2	FR.S.3.1, FR.S.3.2
test-FR-UI-1-1	FR.S.5.1, FR.S.5.5, FR.S.7.3
`test_get_bots_endpoint`	FR.S.7.5
`test_stress_test_endpoint`	FR.S.7.1, NFR.S.1
`test_player_factory_custom_bot`	FR.S.2.5
`test_create_game_all_player_types`	FR.S.2.1, FR.S.2.5
`test_resolve_model_path_absolute_hpc`	NFR.E.3, FR.S.2.2
`test_resolve_model_path_relative`	NFR.E.3, FR.S.2.2
`test_mcts_analysis_endpoint`	FR.S.3.1, FR.S.3.4, FR.S.2.4
`test_mcts_analysis_endpoint_not_found`	FR.S.7.5

Table 2: Functional Test Case Traceability

9.2 Nonfunctional Requirements Trace

Test Case ID	Traced Requirements
test-NFR-P-1	NFR.S.1
test-NFR-U-1	NFR.S.2
test-NFR-U-2	NFR.S.2
test-NFR-M-1	NFR.S.3
test-NFR-I-1	NFR.S.4
test-NFR-D-1	NFR.S.5
test-NFR-D-2	NFR.S.5
test-NFR-AV-1	NFR.S.6

Table 3: Nonfunctional Test Case Traceability

10 Trace to Modules

10.1 Functional Modules Trace

Test Case ID	**Traced Modules**
test-FR-AI-1-1	AI Model, Game State Manager
test-FR-AI-1-2	AI Model, Game State Manager
test-FR-UI-1-1	User Interface, Game State Database
`test_get_bots_endpoint`	Game State Manager
`test_stress_test_endpoint`	Game State Manager, Reinforcement Learning Environment
`test_player_factory_custom_bot`	Reinforcement Learning Environment
`test_create_game_all_player_types`	Reinforcement Learning Environment, Game State Manager
`test_resolve_model_path_absolute_hpc`	AI Model, Reinforcement Learning Environment
`test_resolve_model_path_relative`	AI Model, Reinforcement Learning Environment
`test_mcts_analysis_endpoint`	AI Model, Game State Manager
`test_mcts_analysis_endpoint_not_found`	Game State Manager

Table 4: Functional Test Case to Module Traceability

10.2 Nonfunctional Modules Trace

Test Case ID	**Traced Modules**
test-NFR-P-1	AI Model, Game State Manager, User Interface
test-NFR-U-1	User Interface
test-NFR-U-2	User Interface
test-NFR-M-1	All Modules
test-NFR-I-1	User Interface
test-NFR-D-1	Game State Database
test-NFR-D-2	Game State Database, User Interface
test-NFR-AV-1	Game State Manager, Game State Database

Table 5: Nonfunctional Test Case to Module Traceability

11 Code Coverage Metrics

We have been using Coveralls for coverage tracking as part of our CI/CD pipeline throughout the project. As of the latest report (at time of writing),

our code coverage is 81%. The report can be seen below. It appears to have dipped somewhat recently due to the addition of rev1 code, which has yet to be fully covered.

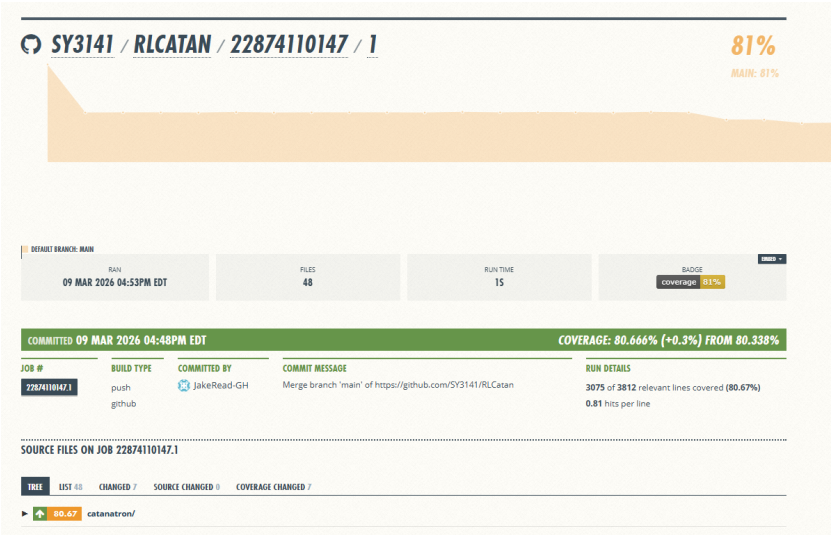

Figure 3: Coveralls Code Coverage Report

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)